Linux Filesystem

1. What is FileSystem?

A file system is a method used by the operating system to organize, store, and manage data on a storage device like a hard disk. It defines how data is stored, accessed, and retrieved

2. Windows vs Linux

Windows path: C:\Users\<user_name>\Downloads

Linux: /home/<user_name>/<file_name>

Windows is case insensitive and Linux is case sensitive

3. Types of File Systems (Common)

ext4 → most common in Linux
xfs → used in enterprise systems
ntfs → Windows
fat32 → USB drives

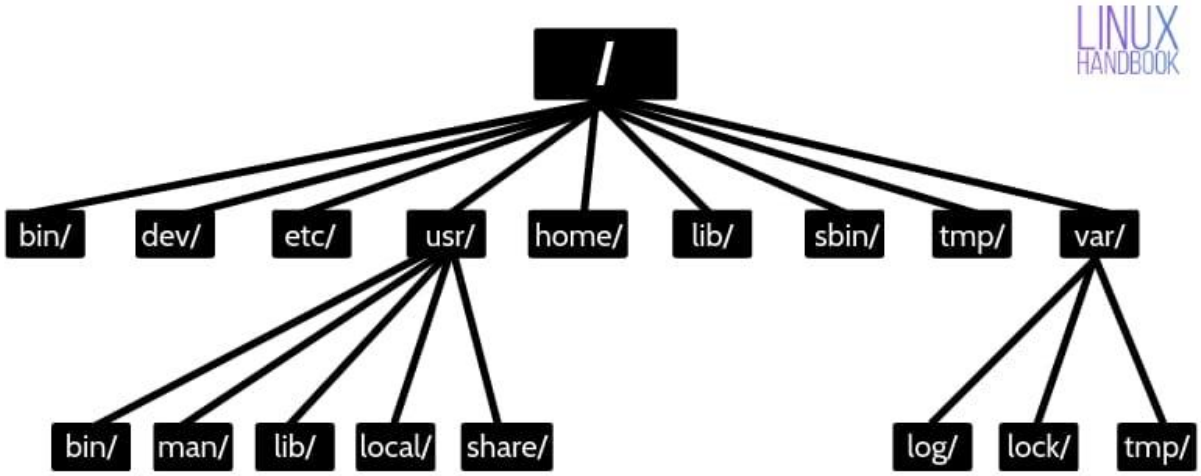
4. Difference between ext4 and XFS

ext4 and XFS are both Linux file systems. ext4 is more general-purpose and widely used, while XFS is designed for high performance and large-scale systems, especially for handling large files and parallel operations.

Key Differences

Feature	ext4	xfs
Type	General purpose	High-performance
Performance	Good	Better for large data
File Size Support	Up to ~16TB	Very large (petabytes)
Resize	Can shrink & grow	Only grow (cannot shrink)
Speed	Moderate	Faster for large files
Journaling	Yes	Yes (more efficient)
Use Case	Default Linux FS	Enterprise / Big data

5. Linux File System Structure



/

Root directory — the top of the filesystem; everything starts from here.

/bin

Essential user commands (like ls, cp, mv) required for basic system operation.

/sbin

System/admin commands used for system maintenance (mainly for root user).

/etc

Stores system-wide configuration files (users, services, network configs).

/home

Contains personal directories for all regular users.

/root

Home directory of the root (admin) user.

/var

Stores variable data like logs, cache, mail, and application data.

/tmp

Temporary files; usually cleared automatically after reboot.

/usr

Contains user-installed applications, libraries, and documentation.

/opt

Used for installing third-party or optional software.

/dev

Contains device files representing hardware (disks, USB, etc.).

/proc

Virtual filesystem showing process and system information (CPU, memory, running processes).

/boot

Contains bootloader files and Linux kernel required for system startup.

/mnt

Temporary mount point for manually mounted filesystems.

/media

Auto-mounted removable media like USB drives and CDs.

/lib & /lib64

Shared libraries required by system programs and binaries.

Users & Groups

1. What is a User?

A **user** is a person or system account that can log into Linux and perform actions.

Example:	**Login Command**
• Root -> admin user	sudo su
• Steve -> normal user	su - <user_name>

2. What is a Group?

A **group** is a collection of users.

Uses:

- To manage permission easily.
- Instead of giving access to each user, you give access to a group.

Example:

- Group: Developers
- Users: ram, shyam , alex

Commands:

```
useradd john      # create user
adduser john      # advanced mode
groupadd dev      # create group
userdel john      #delete user
groupdel dev      #delete group
usermod -aG dev john   # add user to group
gpasswd –d john dev  #remove john from dev group
```

id <user_name>

output: uid=1001(user_name) gid=1004(group_name) groups=1004(group_name),1001(dev)

User ID | **Group ID**

File Permissions

File Permissions (rwx)

Every file/directory has permissions that define: Who can read, write, or execute it

Three Types of Permissions:

Symbol	Meaning	Example
r	Read	View file
w	Write	Edit file
x	Execute	Run file

Three Levels of Access:

Level	Meaning
User (u)	Owner of file
Group (g)	Group members
Others (o)	Everyone else

Each permission has a number:

Permission	Value
r (read)	4
w (write)	2
x (execute)	1

Example1:

-rwxr-xr--

Section	**Meaning**
Rwx	Owner → full access
r-x	Group → read & execute
r--	Others → read only

Easy Trick: Always read in groups of 3:

rwx | r-x | r--

u g o

<u>Change Persmission:</u>

chmod 755 file.txt

7 → rwx

5 → r-x

5 → r-x

<u>Example2:</u>

mkdir /tmp/shared

chmod 1777 /tmp/shared

1 777

| └── Normal permissions (rwx for all)

└─── Special permission (sticky bit)

Owner → full access

Group → full access

Others → full access

So everyone can read, write, create files

What is that 1? (Sticky Bit)

1 = Sticky Bit

What Sticky Bit does

Rule:

Users can create files, but can only delete their own files

<u>Example3:</u>

chmod g+s /tmp/project

This command sets the SGID (Set Group ID) on the /tmp/project directory.

What SGID does?

When SGID is applied on a directory:

All new files and folders created inside will inherit the same group as the directory.

Without SGID

/project → group = dev

User john creates file:

file.txt → group = john

Problem:

Group mismatch

Team cannot access properly

With SGID (After chmod g+s)

/project → group = dev

User john creates file:

file.txt → group = dev

Solution:

All team members (dev group) can access

Ownership

Every file has:

- Owner (user)
- Group

Example1:

-rw-r--r-- 1 john dev file.txt

- Owner = john
- Group = dev

Change Ownership

- chown john file.txt **# change owner**
- chown john:dev file.txt **# change owner + group**

Example2:

-rwxr----- 1 john dev script.sh

- Owner (john) → full access
- Group (dev) → read only
- Others → no access

Example3:

- chown john:dev file.txt
- chmod 770 file.txt
- Owner + group → full access
- Others → no access

Users = who is using system

Groups = collection of users

Permissions = what they can do

Ownership = who controls the file

#Recursive flag: It inherits the command property.

Ex: file path: /home/ubuntu/file.txt

chmod 755 /home/ubuntu #it will change permission for only ubuntu dir

chmod -R 755 /home/ubuntu #it will change permission for ubuntu along with all files & dirs inside ubuntu.

-R → recursive (safe meaning across commands, applicable everywher)

-r → depends on command (cannot use in chmod, chown)

Process Lifecycle

How a process lives & dies

1. What is a Process?

A **process** is equal to a running program

Example:
sleep 100 #command
This creates a process

2. Lifecycle stages

A process goes through different states:

New → Ready → Running → Waiting → Terminated

a. New (Created)

Process is created

Example: you type a command

b. Ready

Waiting for CPU

In queue

c. Running

CPU is executing it

d. Waiting (Sleeping)

Waiting for I/O (disk, network, etc.)

Example:

sleep 10 #command

It is in **waiting state**

e. Terminated

Process finished or killed

How to check process states

ps aux #command

Look at column:

- R → Running
- S→ Sleeping
- Z → Zombie (important)

3. What is Zombie process?

-> Process is dead but still in table

-> Parent didn’t read its status

Kill a process

kill <PID> #command

kill -9 <PID> # force kill command

Foreground vs Background Process

1. Foreground process

-> Runs in terminal and blocks it

Example:

sleep 20 #command

You cannot type anything until it finishes

2. Background process

-> Runs in background → terminal is free

Example:

sleep 100 & #command

Output:

[1] 1234

-> 1 → job id

-> 1234 → PID

Move running process to background

a. Run:

sleep 100 #command

b. Press:

Ctrl + Z

Suspends process

c. Then:

bg #command

Runs in background

Bring back to foreground

fg #command

See background jobs

jobs #command

Summary

Type	Behavior
Foreground	Blocks terminal
Background	Runs freely

System Monitoring (Very Important)

1. Why monitoring?

To check:

-> CPU usage

-> Memory usage

-> Running processes

-> System load

a. **Check CPU usage**

i. top #command

Shows:

-> CPU %
-> Memory
-> Running processes

ii. Better version

htop #command

More user-friendly (install if not present)

b. **Check memory**

free -h #command

Output:

-> total

-> used

-> free

c. **Check processes**

ps aux #command

d. Check specific process

ps aux | grep nginx #command

e. System load

uptime #command

Shows load average

f. Disk usage

df –h #command

g. Real-time process tracking

top –p <PID> #command

htop Command explanation:

1. CPU Colors Explanation

Color	Meaning	Good/Bad
Green	User processes	✅ Normal
Red	Kernel/system	⚠️ Watch
Blue	Nice processes	✅ Fine
Yellow	I/O wait	❌ Disk issue
Purple	Steal time	❌ VM issue
Grey	Idle	✅ Good

Case 1: Mostly Green

-> Normal application load
✓ Healthy system

Case 2: High Red

Kernel busy
Possible causes:

-> Heavy I/O
-> Network issues
-> Driver problem

Case 3: High Yellow

Disk is slow
Common in:
-> Database servers
-> Logging-heavy systems

Case 4: High Purple

Cloud issue

-> Shared CPU overloaded

2. Memory Colors Explained

Green → Used Memory (Applications)
Memory used by:
-> Running programs (Java, Python, etc.)
-> Services (nginx, mysql)
This **is real usage**

Blue → Buffers
Memory used by kernel for:
-> Temporary data
-> Disk operations
✓ Small portion usually
✓ Helps system performance

Yellow → Cache

Memory used for:

-> File caching

-> Frequently accessed data

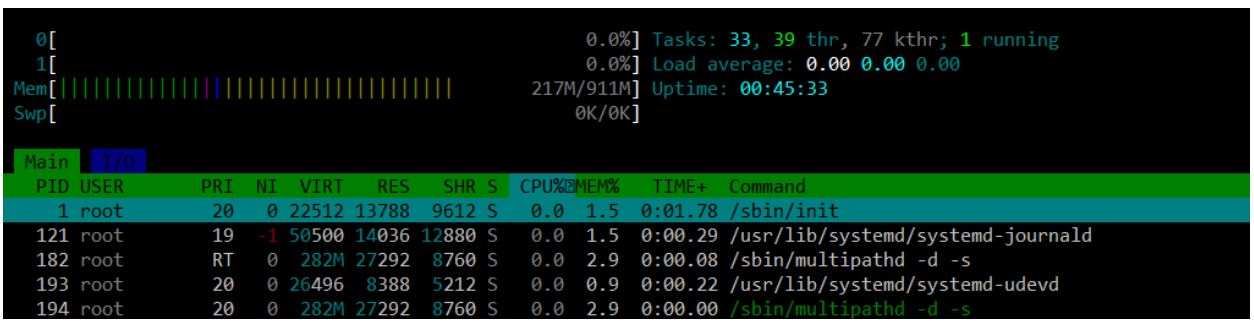
✓ VERY IMPORTANT:

This memory can be freed anytime if needed

Grey → **Free Memory**

-> Completely unused RAM

Color	Meaning	Important?
Green	App memory	✅ Yes
Blue	Buffers	Minor
Yellow	Cache	✅ Very important
Grey	Free	Not critical



a. PID: Process ID

b. User: Name of users

c. PRI(Priority): Lower number -> Higher priority, kernel decides this based on NI

d. NI (Nice value): controls priority manually. Ex: -20(highest priority) , 19(Lowest Priority), 0 (Default Priority)

e. VIRT(Virtual Memory): Total memory used by process including everything (RAM, SWAP, Shared lib, reserved memory). *Note: VIRT is often large and misleading.*

f. RES (Resident memory): Actual RAM used right now.

g. SHR (Shared Memory): Memory shared with other processes. Ex: shared libraries, common resources.

h. S (State): Process state.

top command explanation:

```
top - 11:48:58 up  1:02,  3 users,  load average: 0.00, 0.00, 0.00
Tasks: 116 total,   1 running, 115 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.2 us,  0.3 sy,  0.0 ni, 99.5 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :    911.5 total,    271.6 free,    382.3 used,    416.1 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.    529.2 avail Mem
```

i) **11:48:58** -> current system time

ii) **up 1:02min** -> system running since last reboot

iii) **3 users** -> 2 logged-in users

iv) **load average** -> system load.

0.00, 0.00, 0.00 -> Last 1 min, 5 min, 15 min resp.

Ex: 0.00 -> no load (idle system), 1.00 -> CPU fully used, > CPU cores -> overloaded.

v) **Tasks:**

```
Tasks: 116 total,   1 running, 115 sleeping,   0 stopped,   0 zombie
```

Field	Meaning
total	total processes
running	currently using CPU
sleeping	waiting (normal)
stopped	paused (Ctrl+Z)
zombie	dead but not cleaned

Currently 1 running -> very light load.

115 sleeping -> normal

0 zombie -> healthy system

vi) **CPU usage:**

```
%Cpu(s):  0.2 us,  0.3 sy,  0.0 ni, 99.5 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
```

current -> 99.5 id CPU is 99.5% idlle

Field	Meaning
us	User processes
sy	System (kernel)
ni	Nice (low priority)
id	Idle
wa	Waiting for I/O
hi	Hardware interrupts
si	Software interrupts
st	Stolen (VM usage)

vii) **Memory (RAM):**

```
MiB Mem :    911.5 total,    271.6 free,    382.3 used,    416.1 buff/cache
```

buff/cache is not wasted. Linux is using free RAM as cache to improve performance.

Field	Meaning
total	total RAM
free	unused RAM
used	actively used
buff/cache	cached memory

viii) **Swap:**

```
MiB Swap:      0.0 total,      0.0 free,      0.0 used.    529.2 avail Mem
```

disk used as RAM (backup memory.)
current -> No swap configured, available memory for swap: 592.2 MB

Inode, Kernel, and Daemon

Inodes

Inode is a data structure in Linux that stores metadata about a file such as permissions, owner, size, and disk location. It does not store the file name or actual file content.

Key Points:
- Each file has a unique inode number
- Stores metadata like size, permissions, owner
- Does not store file name or content
- Used by OS to locate file data

Inode Number

An inode number is a unique identifier assigned to every file in a Linux filesystem.
It is used by the system to locate the file's metadata and actual data on disk.
Ex: 123456 file.txt -> inode_number file_name

Commands:
ls -i file.txt
df -i
stat file.txt

Important Concept:
Disk can be full due to space (df -h) or inode exhaustion (df -i)

Kernel

The kernel is the core component of the Linux OS that interacts directly with hardware and manages system resources like CPU, memory, and devices. It acts as a bridge between applications and hardware.

Key Functions:
- CPU management
- Memory management
- Device handling
- System calls execution

Working Flow:
Application → Kernel → Hardware

Example:
When a command like 'ls' is executed, the kernel interacts with the disk and returns the file list.

Daemon

A daemon is a background process that runs continuously to provide system services or perform tasks. It usually starts at boot and runs without user interaction.

Examples:
sshd → SSH service
cron → scheduled tasks
httpd/nginx → web server

Commands:
ps -ef | grep sshd
systemctl status ssh

Kernel vs Daemon

Kernel:
- Core of OS
- Manages hardware
- Runs in kernel space

Daemon:
- Background service
- Provides system/application services
- Runs in user space

Summary

Inodes store file metadata, kernel manages hardware/resources, and daemons provide background services.

Disk Structure

Definition

Disk structure refers to how a storage device is organized into partitions and file systems so the operating system can store and manage data efficiently. It defines how raw disk space is divided and used.

Basic Idea

A disk (like HDD/SSD) is not used directly. It is divided into partitions, and each partition is formatted with a file system.

Disk Structure Flow

Disk → Partition → File System → Files

Important Components

1. Disk
Physical storage device
Example: /dev/sda, /dev/sdb

2. Partition
Logical division of disk
Example: /dev/sda1, /dev/sdb1

3. File System
Organizes data
Example: ext4, xfs

4. Mount Point
Location where disk is attached
Example: /mnt, /data

Example

Suppose you have a new disk /dev/sdb
1. Create partition → /dev/sdb1
2. Format it → ext4 or xfs
3. Mount it → /data
4. Store files

Visual Understanding

```
/dev/sdb (Disk)
  └── /dev/sdb1 (Partition)
      └── ext4 (File System)
          └── \data (Mount Point)
                └──Files & Data
```

Practical Commands

Check disks:
lsblk

Create partition:
fdisk /dev/sdb

Format disk:
mkfs.ext4 /dev/sdb1

Mount disk:
mount /dev/sdb1 /data

Real-Life Analogy

Disk = Land
Partition = Plots
File system = Layout plan
Files = Houses

Key Understanding

- Disk must be partitioned before use
- Partition must be formatted
- Then it can store data

Summary

Disk structure is the organization of a disk into partitions and file systems so data can be stored and accessed properly.

Mounting

Definition

Mounting is the process of attaching a filesystem (disk or partition) to a specific directory so that it becomes accessible to the system. Without mounting, the OS cannot access the data on that disk.

Basic Idea

In Linux, storage is not used directly. It must be mounted to a directory (mount point) before accessing files.

Mounting Flow

Disk → Partition → File System → Mount → Access Files

Example

/dev/sdb1 is mounted to /data.
After mounting, /data shows the contents of /dev/sdb1.

Visual Structure

/dev/sdb1 (disk partition)
↓
mounted on
↓
/data (directory)

Temporary vs Permanent Mount

Temporary Mount:
mount /dev/sdb1 /data (lost after reboot)

Permanent Mount:
Edit /etc/fstab and add:
/dev/sdb1 /data ext4 defaults 0 0

Important Commands

Check mounted disks:
mount
lsblk

Unmounting

umount /data
or
umount /dev/sdb1

Important Points

Directory must exist before mounting.
Cannot unmount if the disk is in use.
Always verify after mounting.

Scenario

If disk is attached but not visible:
1. lsblk
2. mount
3. Check /etc/fstab

Summary

Mounting connects a disk or partition to a directory so the system can access its data. Without mounting, storage devices remain unusable.

LVM (Logical Volume Manager) Basics

Definition

LVM (Logical Volume Manager) is a storage management system that allows flexible allocation and resizing of disk space. It helps combine multiple disks and manage them as a single logical storage.

Why LVM is Used

Normal partitions have fixed size and are hard to resize.
LVM allows easy resizing, combining multiple disks, and better storage management.

LVM Structure

Disk → PV → VG → LV → Filesystem → Mount

Components of LVM

PV (Physical Volume): A physical disk or partition used in LVM (e.g., /dev/sdb).

VG (Volume Group): A pool of storage created by combining multiple PVs (e.g., vg_data).

LV (Logical Volume): A virtual partition created from VG, used like a normal disk (e.g., lv_data).

Visual Structure

```
/dev/sdb + /dev/sdc → VG (vg_data)
                ↓
            LV (lv_data)
                ↓
             /data
```

Practical Steps

1. Create Physical Volume:
pvcreate /dev/sdb

2. Create Volume Group:
vgcreate vg_data /dev/sdb

3. Create Logical Volume:
lvcreate -L 2G -n lv_data vg_data

4. Format:
mkfs.ext4 /dev/vg_data/lv_data

5. Mount:
mkdir /data
mount /dev/vg_data/lv_data /data

Extend Logical Volume

lvextend -L +1G /dev/vg_data/lv_data
resize2fs /dev/vg_data/lv_data

Useful Commands

pvs → Check physical volumes
vgs → Check volume groups
lvs → Check logical volumes

Advantages

Flexible storage management
Easy resizing
Combine multiple disks
Suitable for production environments

Important Points

LVM sits between disk and filesystem.
Logical volumes behave like normal partitions.
Commonly used in servers and cloud environments.

Summary

LVM allows flexible disk management by combining physical storage into logical volumes that can be resized easily.

Practical: AWS EBS Volume Setup Guide (Linux)

Step 1: Verify Attached Volume

Run: **lsblk**

You will see new disk like:

nvme1n1 10G disk

Step 2: Create Partition

Run: **sudo fdisk /dev/nvme1n1**

Inside fdisk:

n → new partition
p → primary
1 → partition number
Enter → default start
Enter → default end
w → save

Step 3: Verify Partition: **lsblk**

Output:

nvme1n1
└─nvme1n1p1

Step 4: Format Filesystem: **sudo mkfs.ext4 /dev/nvme1n1p1**

Step 5: Create Mount Point: **sudo mkdir /data**

Step 6: Mount Volume: **sudo mount /dev/nvme1n1p1 /data**

Step 7: Verify Mount: **df -h**

Step 8: Make Mount Permanent

Edit **fstab:**

sudo nano /etc/fstab

Add line: **/dev/nvme1n1p1 /data ext4 defaults,nofail 0 2**

Test: **sudo mount -a**

Step 9: Store Data

cd /data
echo "Hello AWS Volume" > file.txt

Explanation of fstab Entry

Line: **/dev/nvme1n1p1 /data ext4 defaults,nofail 0 2**

Structure:

<device> <mount_point> <filesystem> <options> <dump> <fsck>

1. Device

/dev/nvme1n1p1

Disk partition

nvme1n1 = disk

p1 = partition 1

2. Mount Point

/data

Location where disk is mounted

Access using: cd /data

3. Filesystem

ext4

Type of filesystem

Common in Linux

4. Options

defaults,nofail

defaults includes:

rw (read/write), suid, dev, exec, auto, nouser, async

nofail: Prevents boot failure if disk is missing

5. Dump

0: Backup option (usually not used)

6. FSCK Order

2

Filesystem check order

1 = root (/)

2 = other filesystems

Practical2: Resize Disk

Step1: increase volume in AWS
10GB → 20GB

Step2: verify new size
lsblk
You'll see disk increased but partition same.

Step3: Extend partition
sudo growpart /dev/nvme1n1 1

Step4: resize filesystem
For ext4:
sudo resize2fs /dev/nvme1n1p1

Done
Check: **df -h**